

Algorithms

Chapter 2 Getting Started

Associate Professor: Ching-Chi Lin

林清池 副教授

chingchi.lin@gmail.com

Department of Computer Science and Engineering
National Taiwan Ocean University

Outline

- ▶ **Insertion sort** 插入性排序
- ▶ **Analyzing algorithms** 分析演算法 (1) 正確性 (2) 時間複雜度
- ▶ **Designing algorithms** 設計演算法 (1) 遞增法 (2) 分別擊破法

The purpose of this chapter

- ▶ Start using frameworks for describing and analyzing algorithms. (1) 描述 (2) 分析演算法
- ▶ Examine two algorithms for sorting: insertion sort and merge sort. 以兩個演算法為例: insertion sort, merge sort
- ▶ Learn how to prove the correctness of an algorithm. 證明演算法的正確性
- ▶ Begin using asymptotic notation to express running-time analysis. 使用漸進式符號描述時間複雜度 (O , Θ , Ω)
- ▶ Learn the technique of “divide and conquer” in the context of merge sort. 學習 divide and conquer 的技巧

Algorithm 精確的計算過程

some value Algorithm → some value
input output

- ▶ **Algorithm**: a well-defined computational procedure that takes some value as input and produces some value as output.
- ▶ Major concerns: 兩個重點
 - ▶ Correctness 正確性
 - ▶ Time complexity 時間複雜度
- ▶ For example: The sorting problem
 - ▶ **Input**: A sequence of n numbers $\langle a_1, a_2, \dots, a_n \rangle$.
 - ▶ **Output**: A permutation $\langle a'_1, a'_2, \dots, a'_n \rangle$ of the input sequence such that $a'_1 \leq a'_2 \leq \dots \leq a'_n$. 由小到大排序
 - ▶ Given the input sequence 31, 41, 59, 26, 41, 58, a sorting algorithm returns as output the sequence 26, 31, 41, 41, 58, 59.

Insertion sort 插入性排序

- Insertion sort: an efficient algorithm for sorting a small number of elements. 在個數不多時是個有效率的演算法

INSERTION-SORT(A)

1. **for** $j \leftarrow 2$ **to** $\text{length}[A]$ 從第2張開始排序
2. **do** $\text{key} \leftarrow A[j]$ 將第j張存起來 → 第j張是將要排序者
3. /* Insert $A[j]$ into the sorted sequence $A[1 \dots j-1]$ */
4. $i \leftarrow j-1$ 從前一張開始問
5. **while** $i > 0$ and $A[i] > \text{key}$ 當前方還有人, 而且比key大
6. **do** $A[i+1] \leftarrow A[i]$ 往右移一個位置
7. $i \leftarrow i-1$ 繼續向前一個比較
8. $A[i+1] \leftarrow \text{key}$ 前方沒人或是沒有比較大

把第j張放到適當位置

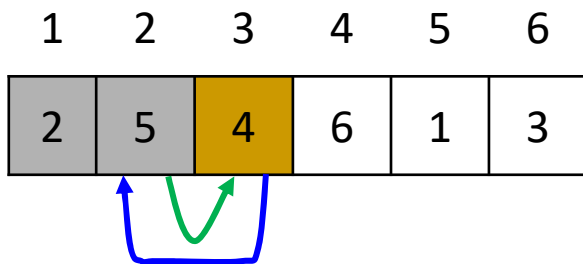


已排序 將要插入進行排序 向前方一個一個比較大小, 調換順序

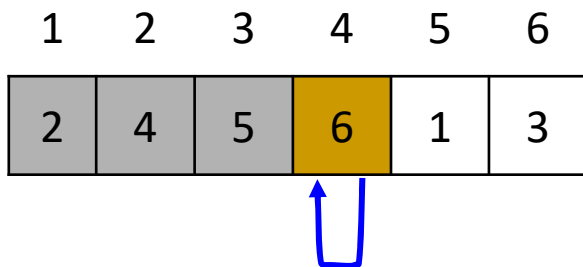


將 2 插入

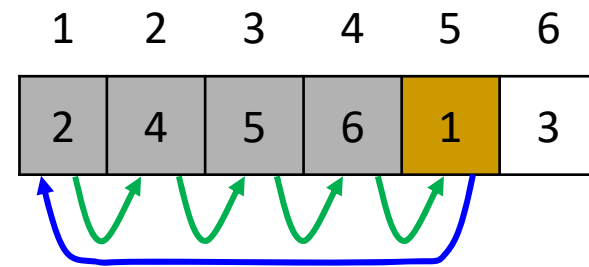
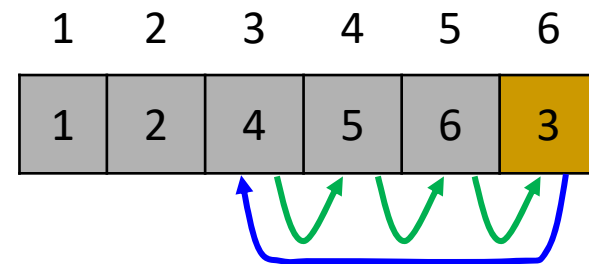
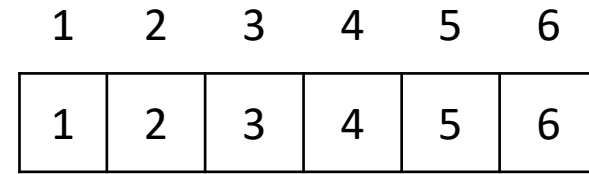
$t_2 = 2 \Rightarrow$ 執行 2 次 (2 v.s. 5, 2 v.s. 5 之前)



$t_3 = 2 \Rightarrow$ 執行 2 次 (4 v.s. 5, 4 v.s. 2) $t_6 = 4 \Rightarrow$ 執行 4 次 (3 v.s. 6, 5, 4, 2)



$t_4 = 1 \Rightarrow$ 執行 1 次 (6 v.s. 5)



$t_5 = 5 \Rightarrow$ 執行 5 次
(1 v.s. 6, 5, 4, 2, 2 之前)

Loop invariant for proving correctness

- ▶ We may use **loop invariants** to prove the correctness.

用 loop invariants 去證明正確性 (共 3 個步驟)

證明敘述
是對的

- (I) ▶ **Initialization:** It is true before the first iteration of the loop.
在第一個 loop 前正確 (在執行 loop 前, 此演算法是正確的)

- (II) ▶ **Maintenance:** If it is true before an iteration of the loop, it remains true before the next iteration.

如果在第 i 個之前正確, 則在經過第 i 個 loop 後依然正確

用敘述
來證明
演算法是對的!

- (III) ▶ **Termination:** When the loop terminates, the invariant gives us a useful property that helps show that the algorithm is correct.

當 loop 結束之後, 可用 invariant 的性質證明正確性

- ▶ Using loop invariants is like mathematical induction.

用 loop invariant 證明 insertion sort 是正確的

Correctness of INSERTION-SORT

經過 loop 後仍保持的性質

- ▶ **Loop invariant:** At the start of each iteration of the **for** loop of lines 1-8, the subarray $A[1..j-1]$ consists of the elements originally in $A[1..j-1]$ but in sorted order. (執行第 j 個 for loop 前, 前 $j-1$ 個已排序好) ↙ 敘述 statement

(I) ▶ **Initialization:** Before the first iteration, $j = 2$. $A[1]$ is trivially sorted.
 $j = 2$, 只有一個, 本身已排序好 Ex: P. 6 圖 - 的 5

(II) ▶ **Maintenance:** Note that the body of the outer **for** loop works by moving $A[j-1]$, $A[j-2]$, $A[j-3]$, ..., and so on by one position to the right until the proper position for $A[j]$ is found.

在 for loop 中, 我們將 $A[j-1]$ $A[j-2]$ $A[j-3]$... 往右移, 且為 $A[j]$ 找到適當位置

(III) ▶ **Termination:** The outer **for** loop ends when j exceeds n , i.e., when $j = n + 1$. Then, $A[1..n]$ is sorted.

當 loop 結束後 $j = n + 1$, 所以根據 loop invariant, $A[1..n]$ 是排序好的
(在執行 j 個 for loop, 前 $j-1$ 個已排序)

Outline

- ▶ Insertion sort
- ▶ **Analyzing algorithms**
- ▶ Designing algorithms

Time complexity of INSERTION-SORT

- Let t_j be the number of times the while loop test for value j .

INSERTION-SORT(A)

	cost	times
1. for $j \leftarrow 2$ to $\text{length}[A]$	c_1	n $j=2 \sim n$ 成功, $j=n+1$ 失敗
2. do $\text{key} \leftarrow A[j]$	c_2	$n-1$ for loop 執行 $n-1$ 次
3. /* Insert $A[j]$ into the sorted sequence $A[1 \dots j-1]$. */	0	$n-1$ 註解不需花費
4. $i \leftarrow j-1$	c_4	$n-1$
5. while $i > 0$ and $A[i] > \text{key}$	c_5	$\sum_{j=2}^n t_j$
6. do $A[i+1] \leftarrow A[i]$	c_6	$\sum_{j=2}^n (t_j - 1)$ 移動次數比測試少1
7. $i \leftarrow i-1$	c_7	$\sum_{j=2}^n (t_j - 1)$
8. $A[i+1] \leftarrow \text{key}$	c_8	$n-1$ 將第 j 個放好

- $T(n) = c_1 n + c_2(n-1) + c_4(n-1) + c_5 \sum_{j=2}^n t_j + c_6 \sum_{j=2}^n (t_j - 1)$

$$+ c_7 \sum_{j=2}^n (t_j - 1) + c_8(n-1)$$

* t_j : 放入第 j 個所要測試的次數 (P.6)

Time complexity of INSERTION-SORT

(I) ▶ **Best-case:** The array is already sorted. 最好的情況: 已排序 Ex: 1, 2, 3, 4, 5, 6

▶ $t_2 = t_3 \dots = t_n = 1$. 每人只測試一次

▶ $T(n) = c_1n + c_2(n-1) + c_4(n-1) + c_5(n-1) + c_8(n-1)$
 $= (c_1 + c_2 + c_4 + c_5 + c_8)n - (c_2 + c_4 + c_5 + c_8) \rightarrow O(n)$

▶ A linear function of n . n 的線性函數 (一次式)

(II) ▶ **Worst-case:** The array is in reverse sorted order. 最差的情況: 倒序

▶ $t_2 = 2, t_3 = 3, \dots, t_n = n$. 每個人都要問到盡頭

Ex: 6, 5, 4, 3, 2, 1

▶ $T(n) = c_1n + c_2(n-1) + c_4(n-1) + c_5\left(\frac{n(n+1)}{2} - 1\right)$

$+ c_6\frac{n(n-1)}{2} + c_7\frac{n(n-1)}{2} + c_8(n-1)$

$= \left(\frac{c_5}{2} + \frac{c_6}{2} + \frac{c_7}{2}\right)n^2 + (c_1 + c_2 + c_4 + \frac{c_5}{2} - \frac{c_6}{2} - \frac{c_7}{2} + c_8)n$
 $- (c_2 + c_4 + c_5 + c_8) \rightarrow O(n^2)$

linear
5n

quadratic
5n²

10	50	500
100	500	50000
10000	50000	500000000

⇒ 成長速率相差很大

▶ A quadratic function of n . n 的二次函數 (平方式)

分析時有3種情形: (I) worse case (II) best case (III) average case
→ 通常只關心最差的

Worst-case and average-case analysis

- ▶ We shall usually concentrate on finding only the worst-case
 - ▶ The worst-case running time gives us a guarantee that the algorithm will never take any longer. 給我們一種保證
 - ▶ For some algorithms, the worst case occurs fairly often. 最差情形常發生
 - ▶ The "average case" is often roughly as bad as the worst case.
average case 與 worst case 差不多
- ▶ For example:
 - ▶ Consider the insertion sort, on average, we check half of the subarray $A[1 \dots j-1]$, so $t_j = j/2$. 如果每個人都只往前測試一半
 - ▶ The average-case running time is still a quadratic function of n .
平均情形仍會是 n 的 2 次函數

Order of growth 成長速率為最要點

- ▶ Another abstraction to ease analysis and focus on the important features. 使用符號來幫助我們(1)更易分析(2)僅關心重點
- ▶ Look only at the leading term of the formula for running time.
 - (I) ▶ Drop lower-order terms. 只看最高項次,其餘項次不看
 - (II) ▶ Ignore the constant coefficient in the leading term. 最高項次係數也不要
- ▶ For example:
 - ▶ The worst-case running time of insertion sort is $an^2 + bn + c$. n 的 2 次函數
 - ▶ Drop lower-order terms $\Rightarrow an^2$. 只看最高項
 - ▶ Ignore constant coefficient $\Rightarrow n^2$. 省去係數
 - ▶ We say that the running time is $\Theta(n^2)$ to capture the notion that the order of growth is n^2 . 表示 insertion sort 的成長速度是 $\underbrace{cn^2}_{\text{常數}}$ (n^2 的常數倍)

Outline

- ▶ Insertion sort
- ▶ Analyzing algorithms
- ▶ **Designing algorithms**

Designing algorithms 設計演算法

- ▶ There are many ways to design algorithms. 設計演算法有很多技巧
- ▶ **Incremental:** 遞增法
 - ▶ For example of insertion sort, having sorted subarray $A[1..j-1]$ and then yielding the sorted array $A[1..j]$.
- ▶ **Divide and conquer** 分別擊破法 (3 steps)
- (I) ▶ **Divide** the problem into a number of subproblems. 分成性質相同的子問題
- (II) ▶ **Conquer** the subproblems by solving them recursively.
 - ▶ If the subproblems sizes are small enough, just solve them in a straightforward manner. 用遞迴方法解決子問題, 如果問題夠小, 用暴力法
- (III) ▶ **Combine** the subproblem solutions to give a solution to the original problem. 將子問題的答案合併, 形成原問題的答案

Merge sort 用 divide and conquer 解決排序

- ▶ **Divide** by splitting into two subarrays $A[p...q]$ and $A[q+1...r]$, where q is the halfway point of $A[p...r]$.

- ▶ **Conquer** by recursively sorting the two subarrays $A[p...q]$ and $A[q+1...r]$. 將陣列分成兩個大小相等的子陣列
分別排序兩個子陣列

- ▶ **Combine** by merging the two sorted subsequences to produce the sorted answer. 將兩個已排序的子陣列合併，
形成一排好的陣列

最左邊 ↘ ↙ 最右邊
MERGE-SORT (A, p, r)

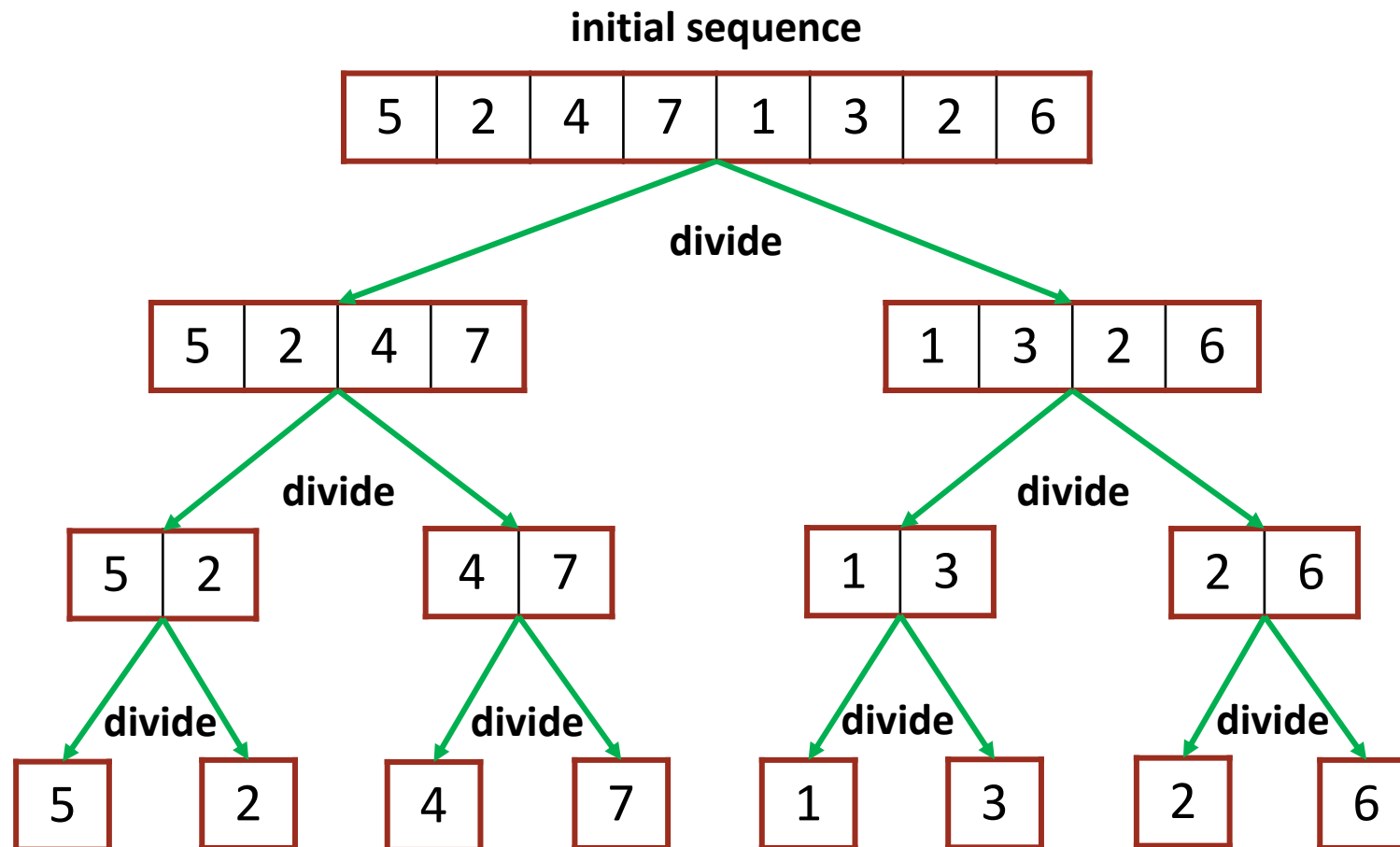
1. if $p < r$ 陣列個數大於1 //Check for base case
2. then $q \leftarrow \lfloor (p+r)/2 \rfloor$ //Divide 中間那一個
3. MERGE-SORT(A, p, q) //Conquer 排左邊
4. MERGE-SORT($A, q+1, r$) //Conquer 排右邊
5. MERGE(A, p, q, r) //Combine 合併

用遞迴的方式
(直接遞迴)

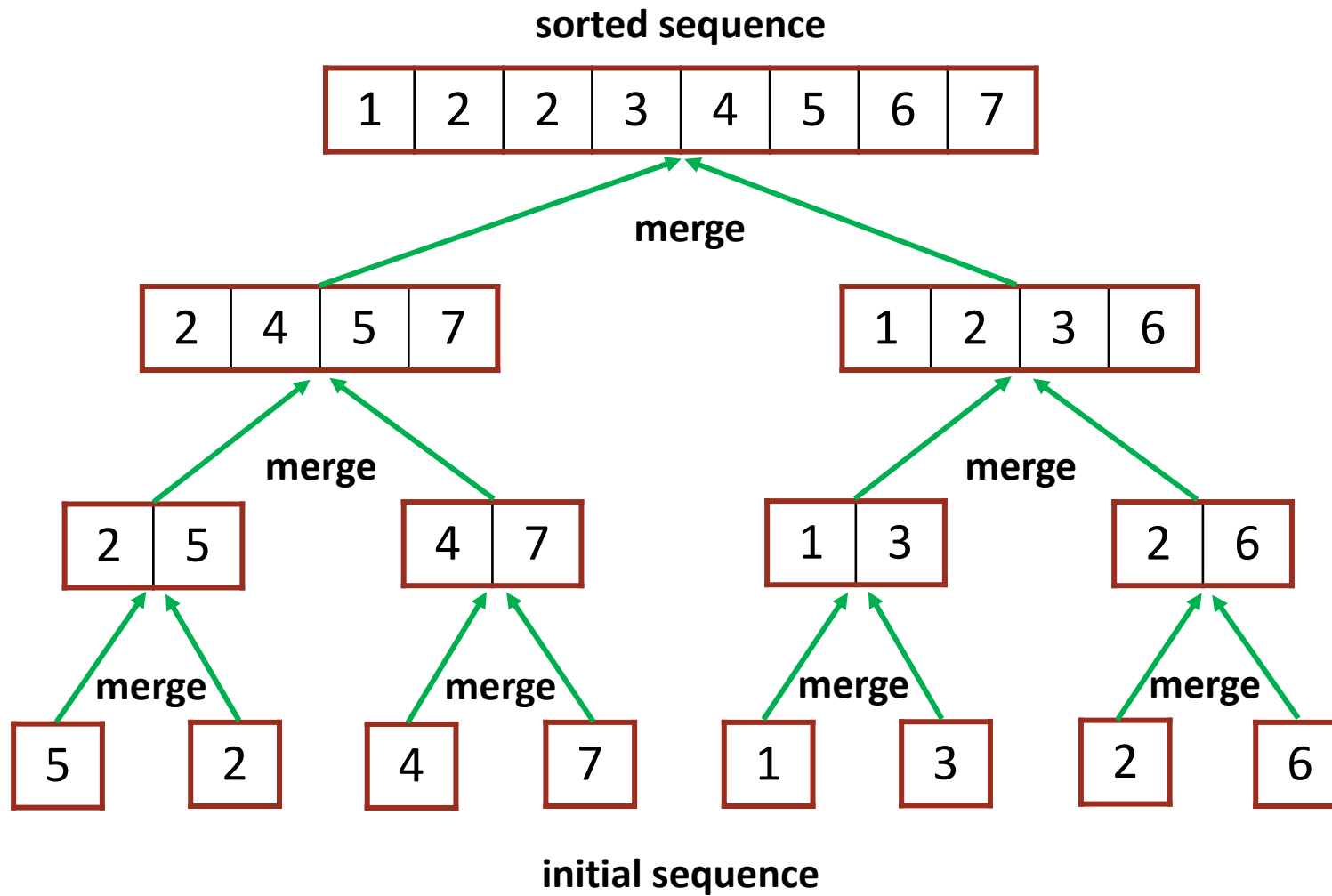
- ▶ **Initial call:** MERGE-SORT($A, 1, n$)
輸入陣列 ↘ ↙ 最右方為 n

最左方為 1

An example for MERGE-SORT



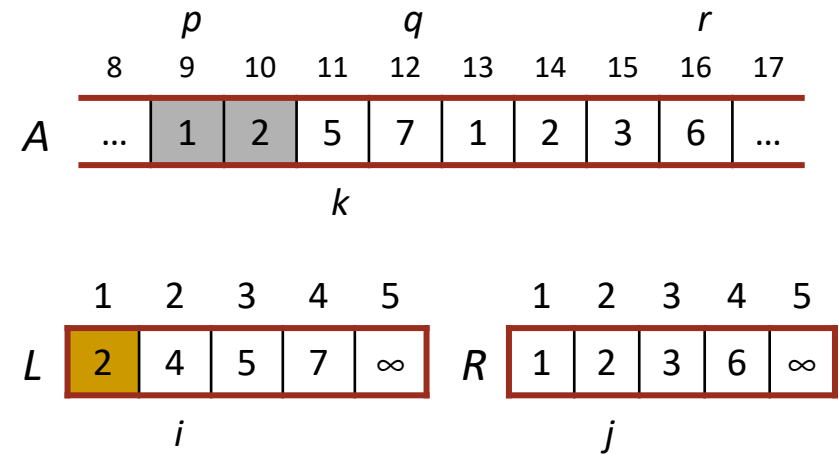
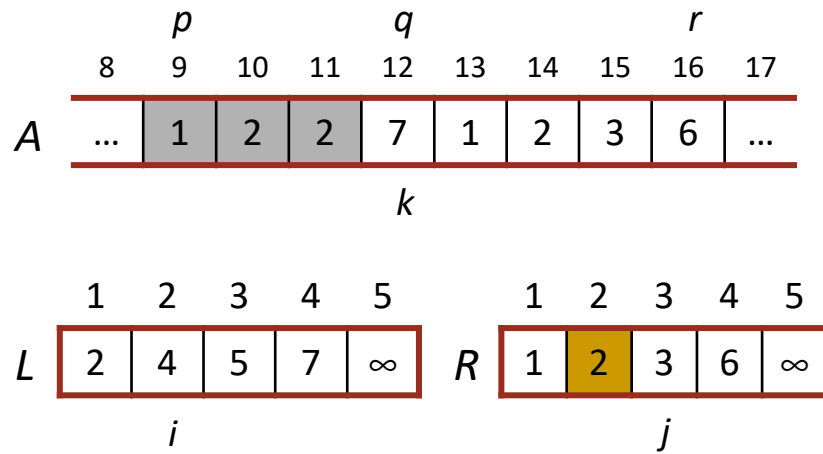
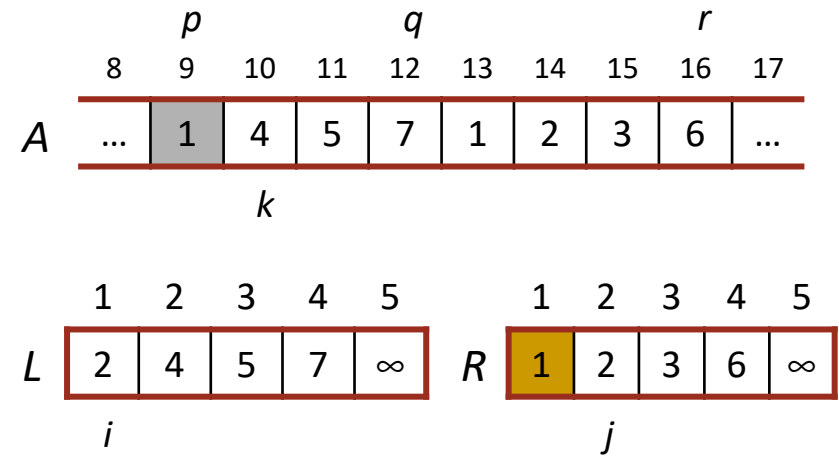
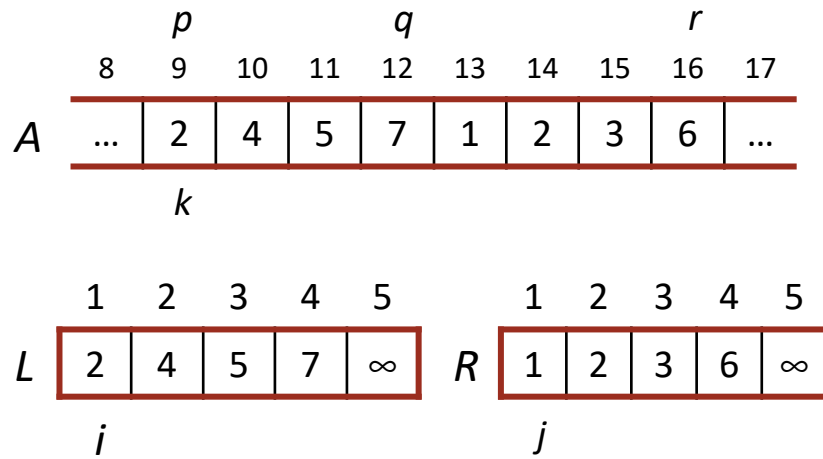
An example for MERGE-SORT

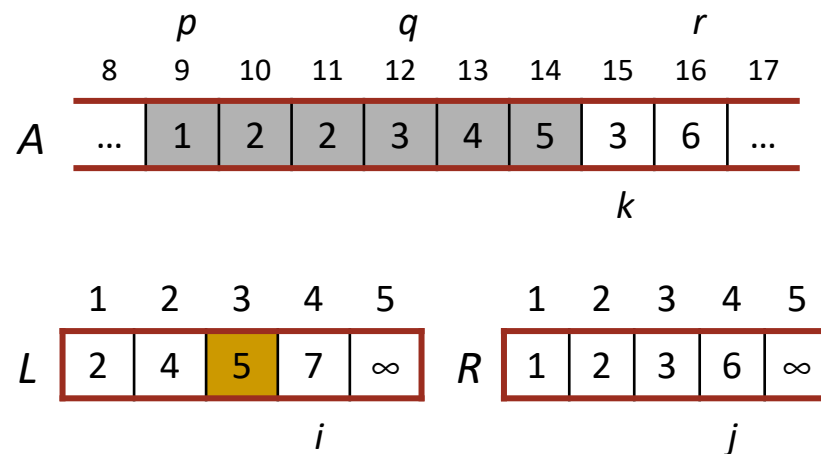
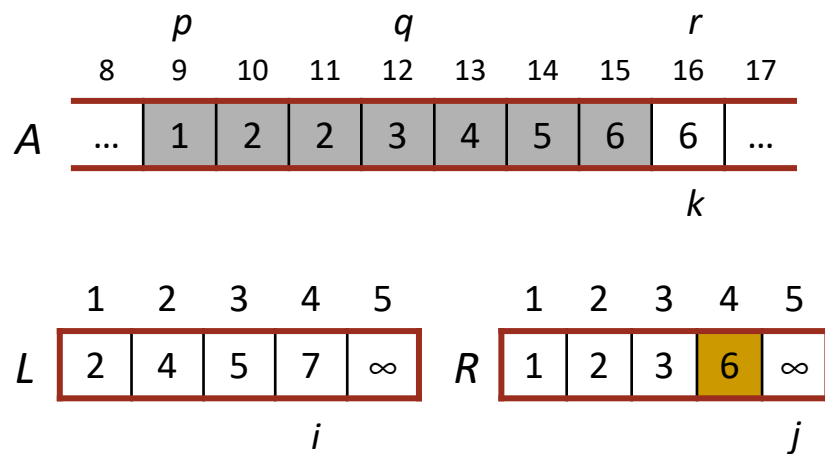
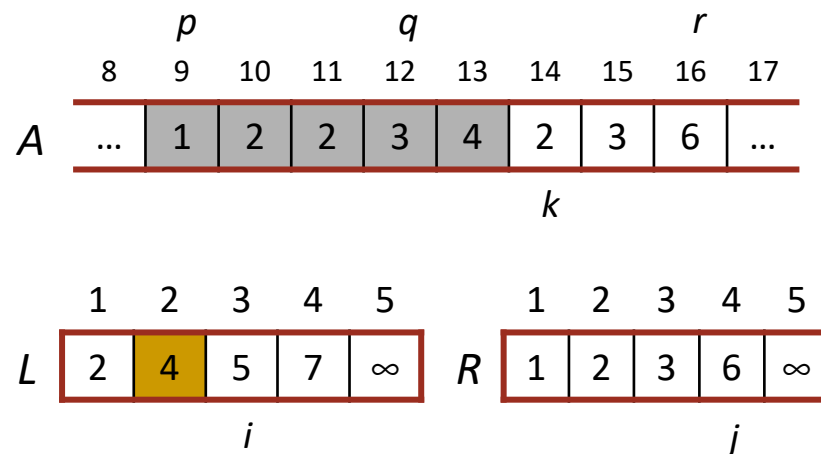
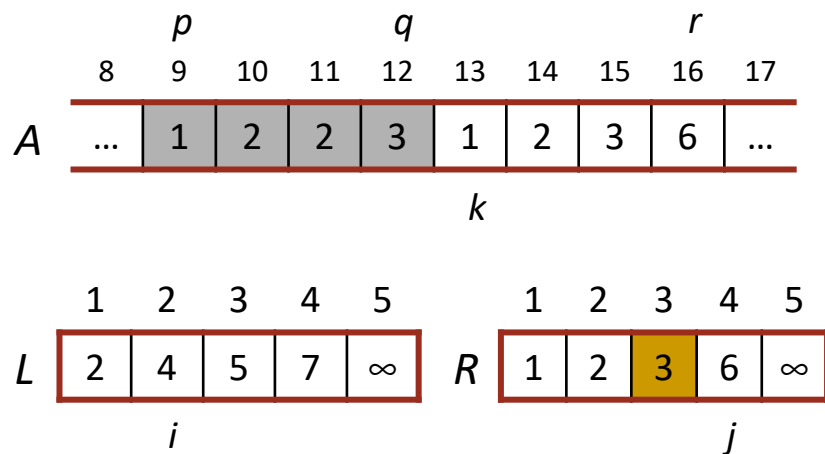


Linear-time merging

MERGE (A, p, q, r)

1. $n_1 \leftarrow q - p + 1$
 2. $n_2 \leftarrow r - q$
 3. create arrays $L[1 \dots n_1 + 1]$ and $R[1 \dots n_2 + 1]$
 4. **for** $i \leftarrow 1$ **to** n_1
 5. **do** $L[i] \leftarrow A[p + i - 1]$
 6. **for** $j \leftarrow 1$ **to** n_2
 7. **do** $R[j] \leftarrow A[q + j]$
 8. $L[n_1 + 1] \leftarrow \infty$; $R[n_2 + 1] \leftarrow \infty$
 9. $i \leftarrow 1$; $j \leftarrow 1$
 10. **for** $k \leftarrow p$ **to** r
 11. **do if** $L[i] \leq R[j]$
 12. **then** $A[k] \leftarrow L[i]$
 13. $i \leftarrow i + 1$
 14. **else** $A[k] \leftarrow R[j]$
 15. $j \leftarrow j + 1$
- $\left. \begin{array}{l} \text{1. } n_1 \leftarrow q - p + 1 \\ \text{2. } n_2 \leftarrow r - q \\ \text{3. create arrays } L[1 \dots n_1 + 1] \text{ and } R[1 \dots n_2 + 1] \end{array} \right\} \begin{array}{l} \Theta(1) \\ n_1: \text{陣列 1 的個數} \\ n_2: \text{陣列 2 的個數} \\ \text{配製 2 個子陣列 } \boxed{L} \quad \boxed{R} \end{array}$
- $\left. \begin{array}{l} \text{4. for } i \leftarrow 1 \text{ to } n_1 \\ \text{5. do } L[i] \leftarrow A[p + i - 1] \\ \text{6. for } j \leftarrow 1 \text{ to } n_2 \\ \text{7. do } R[j] \leftarrow A[q + j] \end{array} \right\} \begin{array}{l} \Theta(n_1 + n_2) \\ \text{將陣列 1 複製到 } L, \text{ 陣列 2 複製到 } R \end{array}$
- $\left. \begin{array}{l} \text{8. } L[n_1 + 1] \leftarrow \infty; R[n_2 + 1] \leftarrow \infty \\ \text{9. } i \leftarrow 1; j \leftarrow 1 \end{array} \right\} \begin{array}{l} \Theta(1) \\ \text{將 } L \text{ 的第 } n_1 + 1 \text{ 放 } \infty, \\ \text{將 } R \text{ 的第 } n_2 + 1 \text{ 放 } \infty \\ \text{(作為邊界條件)} \end{array}$
- $\left. \begin{array}{l} \text{10. for } k \leftarrow p \text{ to } r \\ \text{11. do if } L[i] \leq R[j] \\ \text{12. then } A[k] \leftarrow L[i] \\ \text{13. } i \leftarrow i + 1 \\ \text{14. else } A[k] \leftarrow R[j] \\ \text{15. } j \leftarrow j + 1 \end{array} \right\} \begin{array}{l} \Theta(n_1 + n_2) \\ \text{比大小, 將小的放入 } k \text{ 的位置} \end{array}$





		p			q				r									
	8	9	10	11	12	13	14	15	16	17								
A	...	1	2	2	3	4	5	6	7	...								
																	k	

	1	2	3	4	5		1	2	3	4	5							
L	2	4	5	7	∞		1	2	3	6	∞							
					i						j							

分析 *divide and conquer* 的時間複雜度

Analyzing divide-and-conquer algorithms

- ▶ Use a **recurrence equation** to describe the running time of a divide-and-conquer algorithm.

$$T(n) = \begin{cases} \theta(1) & \text{if } n \leq c, \\ aT(n/b) + D(n) + C(n) & \text{otherwise.} \end{cases}$$

大小為 n 個所需時間 a 個大小為 n/b 的子問題 (解決子問題所需時間)

- ▶ $T(n)$ = the running time on a problem of size n .
- ▶ If $n \leq c$ for some constant c , the solution takes $\Theta(1)$ time.
- ▶ We divide into a subproblems, each $1/b$ the size of the original.
- ▶ $D(n)$ = the time to divide a size- n problem. 分割所需時間 $\lfloor \frac{p+r}{2} \rfloor$
- ▶ $C(n)$ = the time to combine solutions. 合併所需時間 $C(n_1+n_2)$

Analyzing merge sort_{1/2}

- ▶ For simplicity, assume that n is a power of 2.

$$T(n) = \begin{cases} \theta(1) & \text{if } n = 1, \\ 2T(n/2) + \theta(n) & \text{otherwise.} \end{cases}$$

- ▶ The base case occurs when $n = 1$.
- ▶ **Divide**: compute the middle of the subarray, $D(n) = \Theta(1)$.
- ▶ **Conquer**: Recursively solve 2 subproblems, each of size $n/2$
 $\Rightarrow a = 2$ and $b = 2$.
- ▶ **Combine**: MERGE on an n -element subarray takes $\Theta(n)$ time
 $\Rightarrow C(n) = \Theta(n)$.

$$D(n) + C(n) = \theta(1) + \theta(n) = \theta(n)$$

divide + merge
 $\theta(1)$ $\theta(n)$

分成 2 個子問題, 每個子問題的大小是原來的一半

Analyzing merge sort_{2/2}

共 h 層

第 i 層

$$0 \quad 1 \quad 2 \quad \dots \quad h-1$$

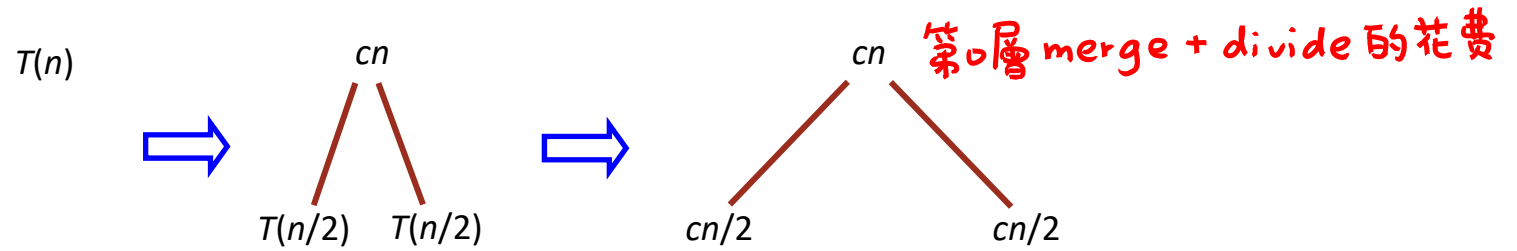
$$n \quad \frac{n}{2} \quad \frac{n}{4} \quad \dots \quad \frac{n}{2^{h-1}} = 1$$

$$2^{h-1} = n \Rightarrow h = \lg n + 1$$

- ▶ Let c be a constant that describes
 - ▶ the running time for the base case
 - ▶ the time per array element for the divide and combine steps.
- ▶ Then, we can rewrite the recurrence as

$$T(n) = \begin{cases} c & \text{if } n = 1, \\ 2T(n/2) + cn & \text{otherwise.} \end{cases}$$

- ▶ The next slide shows successive expansions of the recurrence.
 - ▶ level i : 2^i nodes, each has a cost of $c(n/2^i)$.
So, i th level has a cost of $2^i c(n/2^i) = cn$.
 - ▶ At the bottom level, a tree with h levels has $2^{h-1} = n$ nodes.
Therefore, $h = \lg n + 1$. (h : 層數)
 - ▶ The total cost is $cn(\lg n + 1) = cn \lg n + cn = \Theta(n \lg n)$.
↳ 每一層所需花費



$$\begin{aligned}
 T(n) &= 2T(n/2) + cn \\
 &= 2[2T(n/4) + cn/2] + cn \\
 &= 4T(n/4) + 2 \cdot \frac{cn}{2} + cn
 \end{aligned}$$

